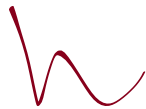


Week 2 – Tutorial Questions

1. Create a class that imitates part of the functionality of the basic data type int. Call the class Int (note different capitalization). The only data in this class is an int variable. Include member functions to initialize an Int to 0, to initialize it to an int value, to display it (it looks just like an int), and to add two Int values. Write a program that exercises this class by creating one uninitialized and two initialized Int values, adding the two initialized values and placing the response in the uninitialized value, and then displaying this result.
2. Student Grade Management: A school needs a system to manage student grades. • Create a class names 'Student' with attributes: studentID, name, and marks (array) for storing marks of 5 subjects. • Implement a function calculateAverage() to return the student's average marks. • Implement another function displayDetails() to show the student's details along with their average marks. Demonstrate how multiple student objects can be created and their details are displayed.
3. Design a **Use case** for e-voting system. Create your own classes and list the attributes and methods for each of them.



WEEK 3-TUTORIAL QUESTION

1. Imagine a tollbooth at a bridge. Cars passing by the booth are expected to pay a 50 cent toll. Mostly they do, but sometimes a car goes by without paying. The tollbooth keeps track of the number of cars that have gone by, and of the total amount of money collected. Model this tollbooth with a class called tollBooth. The two data items are a type unsigned int to hold the total number of cars, and a type double to hold the total amount of money collected. A constructor initializes both of these to 0. A member function called payingCar() increments the car total and adds 0.50 to the cash total. Another function, called nopayCar(), increments the car total but adds nothing to the cash total. Finally, a member function called display() displays the two totals. Make appropriate member functions const.
2. In ocean navigation, locations are measured in degrees and minutes of latitude and longitude. Thus, if you're lying off the mouth of Papeete Harbor in Tahiti, your location is 149 degrees 34.8 minutes west longitude, and 17 degrees 31.5 minutes south latitude. This is Objects and Classes written as 149°34.8' W, 17°31.5' S. There are 60 minutes in a degree. (An older system also divided a minute into 60 seconds, but the modern approach is to use decimal minutes instead.) Longitude is measured from 0 to 180 degrees, east or west from Greenwich, England, to the international dateline in the Pacific. Latitude is measured from 0 to 90 degrees, north or south from the equator to the poles. Create a class angle that includes three member variables: an int for degrees, a float for minutes, and a char for the direction letter (N, S, E, or W). This class can hold either a latitude variable or a longitude variable. Write one member function to obtain an angle value (in degrees and minutes) and a direction from the user, and a second to display the angle value in 179°59.9' E format. Also write a three-argument constructor. Write a main () program that displays an angle initialized with the constructor, and then, within a loop, allows the user to input any angle value, and then displays the value. You can use the hex character constant '\xF8', which usually prints a degree (°) symbol.
3. Create a class that includes a data member that holds a "serial number" for each object created from the class. That is, the first object created will be numbered 1, the second 2, and so on. To do this, you'll need another data member that records a count of how many objects have been created so far. (This member should apply to the class as a whole; not to individual objects. What keyword specifies this?) Then, as each object is created, its constructor can examine this count member variable to determine the appropriate serial number for the new object. Add a member function that permits an object to report its own serial number. Then write a main () program that creates three objects and queries each one about its serial number. They should respond I am object number 2, and so on.

WEEK 4 -TUTORIAL QUESTION

1. Develop a Car Rental System where,

i) Constructor Overloading is used for initializing the class 'Car' with

- Default constructor (sets default values).
- Car model, price per day, and availability status.
- Copy constructor to create a duplicate car.

ii) Function Overloading is used by the customers to book a car in different ways as:

- By specifying the number of days (default price applies).
- By specifying a discount percentage.
- By specifying an extra insurance amount.

iii) Operator Overloading where,

- The + operator should allow combining the total rental prices of two cars
- The > operator should compare which car is more expensive to rent per day.

2. Design a Bank Account class where:

- The account should be initialized with an account number and balance.
- The account should support:
 - Constructor Overloading for creating different types of accounts.
 - Function Overloading to deposit and withdraw using different data types.
 - Operator Overloading to compare two accounts based on balance. Implement a Bank Account class that meets these requirements.

3. Complete the following code to overload the << operator to print the object's value using cout.

```
#include <iostream>
using namespace std;
class Complex {
int real, imag;
public:
Complex(int r, int i) : real(r), imag(i) {}
// Overload << operator here
void display() {
cout << real << " + " << imag << "i" << endl;
}
};
int main() {
Complex c1(3, 4);
cout << c1; // Expected Output: 3 + 4i
return 0;
```

}

Demonstrate the ticket booking and cancellation process with multiple objects.

WEEK 5 -TUTORIAL QUESTION

1. Create a use case diagram and a class diagram for a business situation you're familiar with, whether it's horse trading, software consulting, or dealing in rare comic books.
2. Create an activity diagram for student admission process.
3. Draw a collaboration diagram for an ATM withdrawal system.
4. Draw a use case diagram for a hospital management system.

Tutorial question 6

1.

Manasa is out on a hike with friends. She finds a trail of stones with numbers on them. She starts following the trail and notices that any two consecutive stones' number differ by one of two values. Legend has it that, there is a treasure trove at the end of the trail. If Manasa can guess the value of the last stone, the treasure will be hers. Example

$n=2$ $a=2$ $b=3$

She finds 2 stones and their differences are $a=2$ or $b=3$. We know she starts with a 0 stone not included in her count. The permutations of differences for the two stones are [2,2], [2,3], [3,2] or [3,3]. Looking at each scenario, stones might have [2,4], [2,5], [3,5] or [3,6] on them. The last stone might have any of 4,5 or 6 on its face. Compute all possible numbers that might occur on the last stone given a starting stone with a on it, a number of additional stones found, and the possible differences between consecutive stones. Order the list ascending.

2.

Construct a C++ program to create a class Worker with data members as worker-name, worker_id, no-of-hours-worked, pay-rate, DOJ. Write necessary member functions such as get employee details and Calculate method to calculate and Display Employee Details to display the salary of worker.(Use default value for pay-rate).Display the employee those who are getting more than 20000 per month. Try to automate this software by satisfying all the necessary constraints.

1. In an academic institution, different roles have hierarchical relationships. Implement this scenario using multilevel inheritance in C++:

1. Create a base class Person that contains:
 - A name attribute
 - A function to set and display the name.
2. Create a derived class Teacher that inherits from Person and includes:
 - A subject attribute.
 - A function to set and display the subject.
3. Create a further derived class Professor that inherits from Teacher and includes:
 - A department attribute.
 - A function to set and display the department.
4. Demonstrate multilevel inheritance by:

- Creating an object of Professor.
- Setting values for name, subject, and department.
- Displaying the details of the professor.

Explain how multilevel inheritance works, describe the output, and discuss the advantages and challenges of using multilevel inheritance in object-oriented programming.

2. Student Performance System:

• Develop a student performance management system using hybrid inheritance in C++. Implement the following class hierarchy:

1. Create a base class Student to store student details:
 - Attributes: name, roll number
 - Functions: set and display student details
2. Create a derived class Marks inheriting from Student to manage academic marks:
 - Attribute: marks in three subjects
 - Function: calculate total academic marks
3. Create another derived class Sports to store sports performance:
 - Attribute: sports marks
 - Function: display sports marks
4. Create a final class Result that inherits from both Marks and Sports to compute the overall performance:
 - Function: calculate the total marks (academic + sports) and average

Implementation Requirements:

- Use Hybrid Inheritance (Combination of Multiple and Hierarchical Inheritance).
- Demonstrate the use of virtual base class (if needed) to avoid ambiguity.
- Implement object creation, data input, and output display.
- Show how overriding or function overloading can be applied.

Expected Output:

- Display Student details (name & roll number).
- Display academic marks, total academic marks.
- Display sports marks.
- Display overall result (total and average marks).